

API Pathfinder

Bir API dokümanında doğru “kapiyı” bulan, deneyen ve açıklayan küçük bir yapay zekâ destekli araç — yazılımla uğraşmayanlar için de anlaşılır anlatım

TEK CÜMLEDE

Bir yazılımın “kullanım kılavuzunu” (OpenAPI dokümanı) yüklüyorsun, ne istediğini Türkçe yazıyorsun (“Satışta olan ürünleri listele”); program kılavuzda buna uyan işlemi buluyor, neden seçtiğini açıklıyor ve sen onay verirsen o işlemi gerçekten deneyip sonucunu gösteriyor.

Teknolojiler: Next.js · React · TypeScript · PostgreSQL · pgvector · Docker · Google Gemini
Belgenin sonunda: projeyi adım adım çalıştırma ve ekrandaki her seçeneğin ne işe yaradığı

İçindekiler

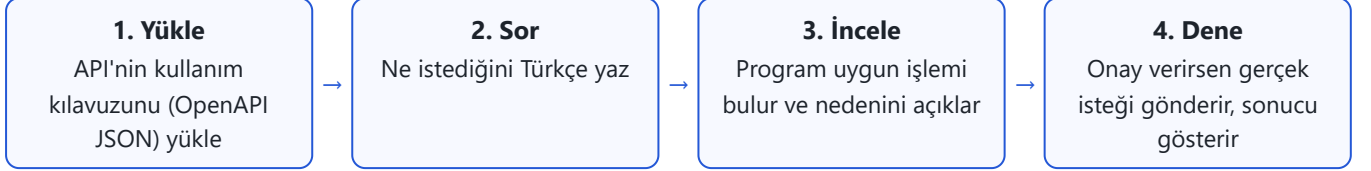
1. Proje ne yapıyor, neden var?
2. Temel kavramlar: yazılım, sunucu, tarayıcı
3. API nedir?
4. HTTP: programların konuşma dili (URL, GET/POST..., durum kodları, JSON)
5. OpenAPI: bir API'nin kullanım kılavuzu
6. Projenin parçaları ve nerede çalıştıkları
7. Docker: neden ve nasıl?
8. Veritabanı: var mı, nerede, içinde ne var?
9. Yapay zekâ kısmı: embedding, pgvector, RAG, araç çağırma (agent)
10. Bir isteğin baştan sona yolculuğu
11. Güvenlik: sistem kendini nasıl koruyor?
12. Projenin bilinçli sınırları
13. Sözlük
14. Projeyi ayağa kaldırma (adım adım)
15. Ekran rehberi: her düğme ne işe yarar, bize ne sunar?
16. Deneme senaryoları ve sorun giderme

Proje ne yapıyor, neden var?

Bugün hemen her uygulama başka uygulamalarla konuşur: bir alışveriş sitesi ödeme sistemine, bir hava durumu uygulaması meteoroloji servisine bağlanır. Bu bağlantı noktalarına **API** denir ve her API'nin, hangi işlemleri sunduğunu anlatan bir **dokümanı** vardır. Bu dokümanlar çoğu zaman uzun, İngilizce ve teknik olur.

API Pathfinder'ın çözdüğü problem şu: “Bu API ile şunu yapmak istiyorum — hangi işlemi, hangi ayarlarla kullanmalıyım?” sorusuna hızlı ve **doğrulanmış** bir cevap vermek.

Kullanıcının gözünden 4 adım



Neyi farklı yapıyor?

- **Uydurmaz.** Dokümanda olmayan bir işlemi önermez. Bulamazsa “bulamadım” der.
- **Bilgi kaynaklarını ayırır.** Ekranda üç renk vardır: **mavi** = dokümanda yazan gerçek, **mor** = yapay zekânın tahmini, **yeşil/kırmızı** = gerçekten denendiğinde ne olduğu.
- **Kendiliğinden hiçbir şey göndermez.** Gerçek istek yalnızca kullanıcı “İsteği dene” düğmesine bastığında gider.
- **Güvenlik sınırları vardır.** Yalnızca izin verilen adreslere, yalnızca “okuma” türü isteklerle gidilir.

BU PROJEDE

Proje içinde küçük bir **örnek mağaza** da var: 20 ürün ve 4 kategoriden oluşan sahte ama gerçekten çalışan bir API. Böylece internete çıkmadan, her seferinde aynı sonuçlarla deneme yapılabilir. Bu mağazanın kullanım kılavuzu `demo/store-openapi.json` dosyasında.

Temel kavramlar: yazılım, sunucu, tarayıcı

İstemci (client) ve sunucu (server)

İnternetteki hemen her şey iki rol üzerine kuruludur: **istemci** bir şey ister, **sunucu** cevap verir. Tarayıcın (Chrome, Edge) bir istemcidir; bir web sitesini açtığına, o sitenin sunucusuna “bana bu sayfayı ver” der.

BENZETME: RESTORAN

Müşteri (istemci) garsona siparişini verir. Mutfak (sunucu) yemeği hazırlar, garson geri getirir. Müşteri mutfağa girmez; sadece menüdeki şeyleri, belirli bir şekilde isteyebilir.

“localhost” ve port

localhost, “bu bilgisayarın kendisi” demektir. Proje senin bilgisayarında çalıştığı için adresi

`http://localhost:3000` 'dir. **Port** (3000, 5433 gibi sayılar), aynı bilgisayardaki farklı programların *kayıp* numarasıdır: bir apartmanın adresi aynıdır, daire numaraları farklıdır.

Port	Kim dinliyor?
3000	API Pathfinder uygulaması (arayüz + sunucu + örnek mağaza API'si)
5433	Docker içindeki PostgreSQL veritabanı (bilgisayarında zaten 5432'de başka bir veritabanı olduğu için 5433 seçildi)

Kullanılan yazılım dilleri ve araçlar

Ad	Ne olduğu (basitçe)	Bu projede görevi
TypeScript	Bir programlama dili. JavaScript'in, hataları erken yakalayan “tipli” hali.	Projenin neredeyse tüm kodu bu dille yazıldı.
Node.js	JavaScript/TypeScript kodunu tarayıcı dışında, bilgisayarda çalıştıran motor.	Sunucu tarafını çalıştırır. <code>npm</code> komutu da onunla gelir.
npm	Node'un paket yöneticisi: başkalarının yazdığı hazır kod parçalarını indirir.	<code>npm install</code> , <code>npm run dev</code> gibi komutlar.
React	Ekrandaki düğme, kutu, tablo gibi parçaları kurmaya yarayan kütüphane.	Arayüzün tamamı.
Next.js	React üzerine kurulu bir “çatı” (framework). Hem web sayfasını hem sunucu kodunu tek projede toplar.	Sayfa + <code>/api/...</code> uç noktaları + örnek mağaza.
PostgreSQL	Çok yaygın, ücretsiz bir veritabanı sistemi.	İçerik aktarılan dokümanları, denemeleri, incelemeleri saklar.
pgvector	PostgreSQL'e “anlam benzerliği ile arama” yeteneği ekleyen eklenti.	Hedefine en yakın işlemleri bulur.
Docker	Programları hazır, taşınabilir kutular (konteyner) içinde çalıştırma aracı.	Veritabanını tek komutla kurup çalıştırır.
Google Gemini	Google'ın yapay zekâ modeli ailesi; internet üzerinden (API ile) kullanılır.	Metni sayılara çevirir (embedding), öneri ve açıklama üretir.
Git / GitHub	Git kodun geçmişini (her değişikliği) kaydeder; GitHub bu geçmişini internette saklayıp paylaşır.	Projenin geçmişi 7 kayıt noktası (commit) olarak saklı.

API nedir?

API (Application Programming Interface — Uygulama Programlama Arayüzü), bir programın *başka programlar* tarafından kullanılmak üzere açtığı kapıdır. İnsanlar bir siteyi fareyle kullanır; programlar ise API ile, belirli kurallara göre yazılmış isteklerle kullanır.

BENZETME: BANKA VEZNESİ

Bankanın kasasına kimse doğrudan gidemez. Vezneye gidersin; “bakiyemi göster” veya “para yatır” gibi **tanımlı işlemlerden** birini, **istenen bilgilerle** (hesap no gibi) istersin; vezne sana bir **sonuç** verir. API bu veznedir.

“API sunmak” ile “API kullanmak” arasındaki fark

	API sunmak	API kullanmak
Rol	Sunucu / mutfak / vezne	İstemci / müşteri
Ne yapar?	Gelen isteği karşılar, cevap üretir	Başka bir sunucuya istek gönderir, cevabı işler
Bu projede	Örnek mağaza: /demo-api/products gibi adresler gelen isteklere ürün listesi döner	Pathfinder: “İsteği dene” düğmesine basınca mağaza API'sine istek gönderir

İLGİNÇ NOKTA

Bu projede iki rol aynı uygulamanın içinde. Pathfinder, kendi içindeki örnek mağazaya sanki dünyanın öbür ucundaki bir API'ymiş gibi davranıyor: aynı güvenlik kurallarıyla, aynı yoldan istek gönderiyor. Böylece gerçek bir API'ye geçmek kolay oluyor.

HTTP: programların konuşma dili

HTTP, tarayıcıların ve programların sunucularla konuştuğu dildir. Her konuşma bir **istek** (request) ve bir **yanıttan** (response) oluşur. Bir isteğin en önemli parçaları: **yöntem** (ne yapmak istiyorsun), **URL** (nereye, neyle), yanıtın en önemli parçaları: **durum kodu** (ne oldu) ve **gövde** (gelen veri).

4.1 URL'nin parçaları

`http://localhost:3000/demo-api/products?status=on_sale`

Parça	Örnek	Anlamı
Şema	<code>http://</code>	Hangi dil/protokol kullanılacak (https = şifreli http)
Sunucu + port	<code>localhost:3000</code>	Hangi bilgisayar, hangi kapı
Taban yol	<code>/demo-api</code>	Bu API'nin sunucudaki "klasörü"
Yol (path)	<code>/products</code>	Hangi kaynak: ürünler
Sorgu (query)	<code>? status=on_sale</code>	Ek filtre: "yalnızca satıştakiler". ? 'den sonra <code>ad=değer</code> çiftleri gelir, birden fazlaysa <code>&</code> ile ayrılır.

Path parametresi ise yolun *içine* yerleşen değerdir: `/products/{id}` kalıbındaki `{id}` yerine 17 yazılınca `/products/17` olur ("17 numaralı ürün"). **Query parametresi** ise yolun *sonuna*, ? 'den sonra eklenir.

4.2 HTTP yöntemleri: GET, POST, PUT, PATCH, DELETE

Yöntem, isteğin **niyetini** söyler. En yaygın beşi:

Yöntem	Niyet	Günlük hayattan	Veriyi değiştirir mi?	Bu projede
GET	Oku / getir	Kütüphaneden kitap kataloğuna bakmak	Hayır	Tek desteklenen yöntem
POST	Yeni bir şey oluştur	Yeni üyelik formu doldurmak	Evet	Gönderilmez (atlanır)
PUT	Bir şeyi tamamen değiştir	Adres kaydını baştan yazmak	Evet	Gönderilmez
PATCH	Bir şeyin bir kısmını değiştir	Sadece telefon numarasını güncellemek	Evet	Gönderilmez
DELETE	Sil	Üyeliği iptal etmek	Evet	Gönderilmez

"GET / SET" HAKKINDA

HTTP'de **"SET" diye bir yöntem yoktur**. Programlamada "get/set" (oku/yaz) ikilisi yaygın olduğu için karışabilir; HTTP'de "yazma" işi **POST, PUT, PATCH** ile, silme işi **DELETE** ile yapılır. Projenin yalnızca **GET** kullanmasının nedeni güvenlidir: GET sadece okur; yanlışlıkla bir şey silmek, sipariş vermek veya para göndermek mümkün olmaz. Örnek dokümandaki `POST /products` işlemi bu yüzden içe aktarmada "Atlandı" olarak listelenir.

4.3 Durum kodları: "ne oldu?"

Her yanıt 3 haneli bir sayıyla başlar. İlk hane durumun türünü söyler: **2xx** başarılı, **3xx** başka yere yönlendirme, **4xx** isteyen tarafın hatası, **5xx** sunucunun hatası.

Kod	Anlamı	Bu projede nerede görülür?
200 OK	İstek başarılı, veri geldi	Satıştaki ürünler, 17 numaralı ürün
302	"Başka adrese git" (yönlendirme)	Pathfinder yönlendirmeleri takip etmez (güvenlik)
400 Bad Request	İstek hatalı (ör. geçersiz değer)	Mağazaya <code>status=cheap</code> gibi tanımsız bir değer gönderilirse
401 / 403	Kimlik doğrulama gerekli / izin yok	Anahtar isteyen API'ler — bu yüzden <code>/orders</code> çalıştırılmaz
404 Not Found	Böyle bir şey yok	999 numaralı ürün
429 Too Many Requests	Çok fazla istek / kota doldu	Gemini ücretsiz kotası dolarsa
500 / 503	Sunucu hatası / servis kullanılamıyor	Veritabanı kapalıysa ya da Gemini anahtarı yoksa

ÖNEMLİ AYRIM

200 almak, "istediğim oldu" demek değildir. 200 yalnızca "sunucu isteği kabul etti ve cevap verdi" demektir. Gelen veri gerçekten senin hedefine uyuyor mu, ona bakmak gerekir. Pathfinder bu yüzden 200'ün altına "isteğin çalıştığını gösterir, hedefini karşıladığını değil" notunu yazar.

4.4 JSON: verinin biçimi

JSON, programların veri alışverişinde kullandığı, insanların da okuyabildiği bir metin biçimidir. Süslü parantez { } bir "nesne"yi (etiketli bilgiler), köşeli parantez [] bir "liste"yi gösterir. Örnek mağazadan gelen gerçek yanıt:

```
{
  "id": 17,
  "name": "Wireless Mouse",      ← ürünün adı
  "categoryId": 4,               ← 4 = Electronics kategorisi
  "price": 127.5,
  "status": "on_sale"           ← satışta / indirimde
}
```

OpenAPI: bir API'nin kullanım kılavuzu

OpenAPI, bir API'nin hangi işlemleri sunduğunu, her işlemin hangi bilgileri istediğini ve ne döndürdüğünü anlatan **standart bir doküman biçimidir**. Bu doküman JSON (ya da YAML) ile yazılır. Bir restoranın menüsü gibidir: hangi yemekler var, hangi seçenekleri var, porsiyon nasıl gelir.

Örnek mağaza dokümanında ne var?

İşlem (operationId)	İstek	Ne yapar	Parametre	Pathfinder'da durumu
listProducts	GET /products	Ürünleri listeler	status (isteğe bağlı): on_sale / in_stock / out_of_stock	Çalıştırılabilir
getProductById	GET /products/{id}	Tek bir ürünü getirir	id (zorunlu, tam sayı)	Çalıştırılabilir
listCategories	GET /categories	Kategorileri listeler	—	Çalıştırılabilir
listOrders	GET /orders	Siparişleri listeler	API anahtarı ister	Çalıştırılmaz (kimlik doğrulama gerekir)
createProduct	POST /products	Ürün oluşturur	—	Atlanır (yalnızca GET)

Dokümandaki bazı terimler

- **operationId**: işlemin benzersiz adı (ör. `listProducts`). Pathfinder her şeyi bu adla takip eder.
- **servers**: API'nin hangi adreste çalıştığı. Örnekte `http://localhost:3000/demo-api`.
- **parameters**: işlemin istediği bilgiler; `in`: "query" veya `in`: "path" nerede gönderileceğini söyler, `required` zorunlu olup olmadığını, `schema` türünü (sayı, metin...), `enum` izin verilen değer listesini.
- **\$ref**: "bu bilgiyi dokümanın şu başka yerinden al" bağlantısı. Tekrarı önler. Pathfinder yalnızca dokümanın *kendi içindeki* bağlantıları (`#/...` ile başlayanlar) çözer; internetteki başka dosyalara giden bağlantıları **indirmez**, açık bir hata verir.
- **security**: işlemin kimlik doğrulama (anahtar, şifre) gerektirdiğini söyler. Böyle işlemler Pathfinder'da "çalıştırılmaz" olur.

NEDEN KILAVUZA BU KADAR GÜVENİYORUZ?

Pathfinder'in "uydurmama" garantisi buradan gelir: istek adresi **asla** kullanıcıdan veya yapay zekâdan alınmaz; her zaman bu kılavuzdaki sunucu adresi + işlemin yolu + kılavuza göre kontrol edilmiş parametrelerle program tarafından oluşturulur.

Projenin parçaları ve nerede çalıştıkları

Proje, birbirleriyle konuşan birkaç parçadan oluşur. Hepsi senin bilgisayarında çalışır; yalnızca yapay zekâ (Gemini) Google'ın sunucularındadır.

① **Arayüz (tarayıcıda)** — Gördüğün sayfa. Düğmelere bastığında aşağıdaki sunucuya istek gönderir. Dosya: `src/app/page.tsx`, `src/components/`

② **Uygulama sunucusu (Next.js, port 3000)** — Asıl işi yapan beyin. Dokümanı ayrıştırır, veritabanına yazar, yapay zekâyla konuşur, güvenlik kontrollerini yapar, onaylı isteği gönderir. Arayüzün kullandığı kendi uç noktaları vardır: `/api/import`, `/api/search`, `/api/investigate`, `/api/try` ... Dosyalar: `src/app/api/`, `src/lib/`

③ **Örnek mağaza API'si (aynı sunucuda, /demo-api/...)** — Denemelerin hedefi. Gerçek HTTP yanıtları döndürür (200, 400, 404). Dosyalar: `src/app/demo-api/`, `src/lib/demo-store-data.ts`

④ **Veritabanı (PostgreSQL + pgvector, Docker içinde, port 5433)** — Kalıcı hafıza: dokümanlar, işlemler, anlam vektörleri, deneme ve inceleme kayıtları. Dosyalar: `docker-compose.yml`, `db/init.sql`

⑤ **Google Gemini (internet üzerinden)** — İki iş yapar: metni anlam vektörüne çevirmek (embedding) ve öneri/açıklama üretmek. Kullanmak için **API anahtarı** gerekir; anahtar `.env.local` dosyasında durur.

Kim kiminle konuşur?

```

Tarayıcı —(1)—> Uygulama sunucusu (localhost:3000)
                    |
                    |—(2)—> Veritabanı (Docker, localhost:5433)
                    |—(3)—> Google Gemini (internet)
                    |—(4)—> Örnek mağaza API'si (localhost:3000/demo-api) ← yalnızca
“İsteği dene” ile
  
```

Ayarlar dosyası: `.env.local`

Programın gizli ve ortama özel ayarları bu dosyada durur. **Git'e (dolayısıyla GitHub'a) asla gönderilmez**; herkese açık `.env.example` dosyası ise yalnızca şablondur (içinde gerçek şifre yoktur).

Ayar	Ne işe yarar
<code>POSTGRES_PASSWORD</code> , <code>DATABASE_URL</code>	Veritabanı şifresi ve bağlantı adresi
<code>GEMINI_API_KEY</code>	Google yapay zekâsını kullanma anahtarı (kişiye özel, gizli)
<code>GEMINI_MODEL</code>	Öneri üreten model (varsayılan <code>gemini-3.5-flash-lite</code>)
<code>GEMINI_EMBEDDING_MODEL</code> , <code>EMBEDDING_DIM</code>	Anlam vektörü modeli ve vektörün uzunluğu (768)
<code>ALLOWED_API_ORIGINS</code>	İstek gönderilmesine izin verilen adresler listesi (varsayılan yalnızca <code>http://localhost:3000</code>)
<code>REQUEST_TIMEOUT_MS</code> , <code>MAX_RESPONSE_BYTES</code>	Bir isteğin en fazla ne kadar bekleneceği (5 sn) ve en fazla ne kadar veri okunacağı (64 KB)

Docker: neden ve nasıl?

Bir veritabanını bilgisayara “normal yoldan” kurmak; doğru sürümü bulmak, kurulum sihirbazları, ayarlar, eklentiler (pgvector) demektir ve her bilgisayarda biraz farklı gider. **Docker** bu işi tek komuta indirir.

BENZETME: NAKLİYE KONTEYNERİ

Gemilerdeki standart konteynerler içinde ne olursa olsun aynı vinçle taşınır. Docker da bir programı, çalışması için gereken her şeyle birlikte standart bir kutuya koyar; bu kutu Windows'ta, Mac'te, sunucuda aynı şekilde çalışır.

Docker terimleri

Terim	Anlamı	Bu projede
İmaj (image)	Hazır paketlenmiş program şablonu (“kalıp”)	pgvector/pgvector:pg17 — PostgreSQL 17 + pgvector eklentisi hazır
Konteyner (container)	İmajdan başlatılmış, çalışan kopya	pathfinder-db
Birim (volume)	Konteyner silinse bile kalan kalıcı disk alanı	api-pathfinder_pgdata — veritabanı dosyaları burada
Port eşleme	Konteynerin içindeki kapıyı bilgisayardaki bir kapıya bağlamak	127.0.0.1:5433 → 5432 : bilgisayardaki 5433, kutunun içindeki 5432'ye gider; yalnızca bu bilgisayardan erişilebilir
Docker Compose	Konteynerleri bir ayar dosyasıyla tarif edip tek komutla başlatma	docker-compose.yml
Healthcheck	“Konteyner gerçekten hazır mı?” kontrolü	Birkaç saniyede bir pg_isready ile veritabanı yoklanır
Docker Desktop	Windows/Mac'te Docker'ı çalıştıran uygulama	Açık olmalı; görev çubuğunda balina simgesi

npm run db:up dediğinde ne olur?

1. Docker, docker-compose.yml dosyasını okur; şifreyi .env.local 'dan alır.
2. İmaj bilgisayarda yoksa internetten indirir (yalnızca ilk sefer).
3. pathfinder-db konteynerini başlatır, pgdata birimini bağlar.
4. **İlk açılışta** (birim boşa) db/init.sql dosyasını çalıştırır: pgvector eklentisini açar ve 4 tabloyu oluşturur.
5. Healthcheck “healthy” der; uygulama artık bağlanabilir.

Bunun gerçekten olduğunu npm run db:logs ile görebilirsin: çıktıda running /docker-entrypoint-initdb.d/001-init.sql, ardından CREATE EXTENSION, 4 kez CREATE TABLE ve ready to accept connections satırları görünür.

NEDEN SADECE VERİTABANI DOCKER'DA?

Uygulamanın kendisi (Next.js) npm run dev ile doğrudan çalışıyor; böylece kodda yaptığın değişiklik anında ekrana yansıyor. Docker'a yalnızca kurulumu zahmetli olan parça (veritabanı + eklenti) konuldu.

Veritabanı: var mı, nerede, içinde ne var?

Evet, var. Veritabanı, bilgiyi düzenli ve kalıcı şekilde saklayan bir programdır. Bilgiler **tablolarda** durur; tabloyu bir Excel sayfası gibi düşünebilirsin: **sütunlar** bilgi türünü (ad, adres...), **satırlar** tek tek kayıtları tutar.

Nerede?

- **Program olarak:** Docker içindeki `pathfinder-db` konteynerinde çalışan PostgreSQL 17.
- **Adres:** `localhost:5433`, veritabanı adı `pathfinder`, kullanıcı `pathfinder`, şifre `.env.local` 'da.
- **Dosyalar diskte:** Docker'ın `api-pathfinder_pgdata` biriminde (Docker Desktop'ın kendi disk alanı içinde). Proje klasöründe değil; bu yüzden bilgisayarı kapatıp açsan da, `db:down` desen de veriler kalır. Yalnızca `npm run db:reset` siler.

4 tablo ve görevleri

Tablo	Ne saklar?	Önemli sütunlar
<code>api_documents</code>	Her içe aktarılan OpenAPI dokümanı (bir satır = bir doküman)	başlık, sürüm, sunucu adresi, dokümanın ham hali
<code>operations</code>	Dokümanlardaki her GET işlemi	<code>operation_id</code> , yol, özet, açıklama, parametreler, <code>embed_text</code> (anlamı çıkarılan metin), <code>embedding</code> (768 sayılık anlam vektörü)
<code>try_runs</code>	Her "İsteği dene" denemesi	parametreler, sonuç türü (<code>sent</code> / <code>needs_input</code> / <code>not_executable</code>), gönderilen URL, durum kodu, gelen yanıt
<code>investigations</code>	Her yapay zekâ incelemesi	hedef, mod (<code>rag</code> / <code>agent</code>), <code>trace</code> (modelin istediği araçlar ve sonuçları), cevap

```
api_documents (1) —< operations (çok) —< try_runs (çok)
      |
      |——< investigations (çok)
“Bir dokümanın çok işlemi olur; bir işlem çok kez denenebilir.”
```

Veritabanına kendin bakmak

`npm run db:psql` komutu veritabanına bir sohbet penceresi açar. Burada **SQL** (veritabanı sorgu dili) yazarsın:

```
select id, title from api_documents;           -- hangi dokümanlar var?
select operation_id, path from operations;      -- hangi işlemler kayıtlı?
select id, state, url, http_status from try_runs; -- hangi denemeler yapıldı?
select id, mode, goal from investigations;      -- hangi incelemeler yapıldı?
\q                                              -- çıkış
```

GÜVENLİK NOTU: PARAMETRELİ SORGU

Doküman içeriği güvenilmezdir; içinde kötü niyetli SQL parçası olabilir ("SQL injection"). Bu yüzden kod, değerleri sorgu metnine asla yapıştırmaz; `$1`, `$2` gibi yer tutucular kullanır ve değerleri ayrı gönderir. Veritabanı onları her zaman "veri" olarak görür, "komut" olarak değil.

Yapay zekâ kısmı

9.1 Embedding: metni “anlam koordinatına” çevirmek

Embedding, bir metni uzun bir sayı listesine (burada **768 sayı**) çeviren işlemdir. Bu sayılar, metnin anlamını çok boyutlu bir “anlam haritası” üzerindeki bir noktanın koordinatları gibi temsil eder. Anlamca benzer metinler haritada birbirine yakın düşer.

BENZETME: HARİTA

“Satışta olan ürünleri listele” ile “indirimdeki ürünleri göster” cümleleri farklı kelimeler kullansa da haritada yan yana düşer; “hava durumu” ise uzakta bir yerdedir. Bilgisayar kelimeleri değil, noktalar arasındaki **mesafeyi** karşılaştırır.

Pathfinder, içe aktarmada her işlemin açıklamasını bir kez embedding'e çevirip saklar. Sen bir hedef yazınca, hedefin de embedding'i alınır ve kayıtlı noktalarla karşılaştırılır.

9.2 pgvector: “en yakın hangisi?”

pgvector, PostgreSQL'in bu 768 sayılı vektörleri saklamasını ve aralarındaki mesafeyi hızlıca ölçmesini sağlar. Mesafe ölçüsü **kosinüs benzerliği**dir: iki ok aynı yöne bakıyorsa benzerlik 1'e yakındır, farklı yönlerde bakıyorsa küçülür.

EN YAKIN = DOĞRU DEĞİL

pgvector her zaman bir “en yakın” sonuç verir, “hiçbiri uymuyor” demez. Gerçek bir ölçüm: dokümanda karşılığı olmayan “Müşteri yorumlarını listele” hedefinde en yakın sonuç `listOrders` **0.643** aldı; oysa doğru bir eşleşme olan “17 numaralı ürünü getir” → `getProductById` yalnızca **0.622** aldı. Yani tek başına puana bakıp karar vermek yanlış olurdu. Bu yüzden son kararı yapay zekânın yorumu ve ardından **kodun kontrolü** verir.

Hedefini yaz

Müşteri yorumlarını listele

Ham arama (yalnızca pgvector)İncele (RAG)

İncele (Agent + araçlar)

Arama ve inceleme hedef API'ye istek göndermez.

HAM PGVECTOR SONUÇLARI (KOSİNÜS MESAFESİNE GÖRE)

#	İşlem	Benzerlik	Mesafe
1	listOrders GET /orders ► embedding metni	0.643	0.357
2	listProducts GET /products ► embedding metni	0.610	0.390
3	listCategories GET /categories ► embedding metni	0.583	0.417
4	getProductById GET /products/{id} ► embedding metni	0.562	0.438

En yakın sonuç otomatik olarak doğru sonuç değildir; bu tablo yalnızca anlamsal yakınlığı gösterir.

"Ham arama" ekranı: sadece pgvector sıralaması, yapay zekâ yorumu yok. En üstteki sonuç yanlış bir eşleşme.

9.3 Dil modeli (Gemini)

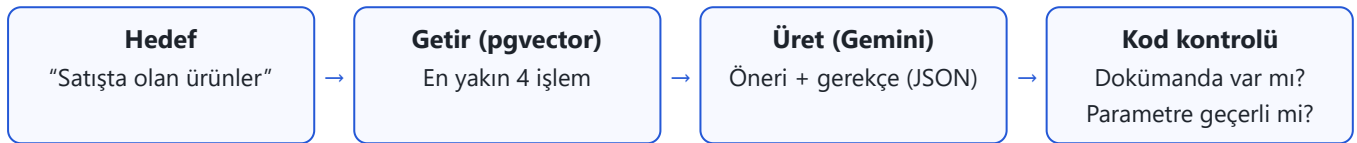
Dil modeli (LLM), metin okuyup metin üreten yapay zekâdır. Pathfinder'da `gemini-3.5-flash-lite` modeli, bulunan aday işlemleri okuyup "hedefe en uygun olan şu, parametresi şu olmalı, çünkü..." diye gerekçeli bir öneri yazar. Model güçlüdür ama **yanılabilir**; bu yüzden ekranda mor renkle "doğrulanmamış çıkarım" diye işaretlenir.

9.4 RAG: "önce bul, sonra sor"

RAG (Retrieval-Augmented Generation — Getirme ile Güçlendirilmiş Üretim), modele soruyu sormadan önce ilgili bilgiyi **bulup** soruyla birlikte vermektir. Model kendi hafızasından değil, önüne konan belgelerden cevap verir.

BENZETME: AÇIK KİTAP SINAVI

Öğrenciye (modele) bütün kütüphane verilmez; kütüphaneci (pgvector) konuyla en ilgili 4 sayfayı bulup önüne koyar, öğrenci cevabını bu sayfalara dayanarak yazar. "Getirilen bağlam" bölümünde tam olarak hangi 4 sayfanın verildiğini görebilirsin.



RAG modunda adımlar **sabittir**: her seferinde önce ara, sonra cevapla.

9.5 Araç çağırma (function calling) ve agent

Function calling, modele "kullanabileceğin araçlar şunlar" diye bir liste vermektir. Model bir aracı kendisi **çalıştıramaz**; sadece "şu aracı, şu bilgiyle çalıştırmak istiyorum" diye **istek** yazar. Aracı çalıştıran, izin verip

vermeyeceğine karar veren Pathfinder'ın kodudur. Sonuç modele geri verilir ve model bir sonraki adımına karar verir. Bu döngüye **agent** (ajan) denir.

Araç	Ne yapar	Modele verildi mi?
<code>search_endpoints(query)</code>	Verilen metinle anlam araması yapar, en yakın 4 işlemi döndürür	Evet
<code>inspect_endpoint(operation_id)</code>	Bir işlemin dokümandaki tüm detayını döndürür	Evet
<code>try_get_request(operation_id, parameters)</code>	Gerçek GET isteğini gönderir	Hayır. Yalnızca sen "İsteği dene"ye basınca çalışır

GERÇEK BİR AGENT ÇALIŞMASI (SENARYO 1)

Tur 1: Model: "search_endpoints('Satışta olan ürünleri listele') çalıştır." → Sunucu arar: listProducts 0.714, listCategories 0.643...

Tur 2: Model en yüksek puana körü körüne güvenmez: "inspect_endpoint('listProducts') çalıştır." → Sunucu detayı verir: "status parametresi var, çalıştırılabilir."

Tur 3: Model artık araç istemez, son cevabını yazar: "listProducts, status=on_sale."

Sınır: en fazla **4 araç adımı**. Model sonsuza kadar araç istese bile kod 4'te durdurur.

RAG ile agent farkı: RAG'de yol baştan çizilidir. Agent'ta ise model bir aracın sonucuna bakıp sonraki adımı *kendisi* seçer; örneğin ilk arama zayıf çıkarsa İngilizce anahtar kelimelerle tekrar arar ya da bir adayın detayına bakar.

Bir isteğin baştan sona yolculuğu

“Satışta olan ürünleri listele” yazıp **İncele (Agent + araçlar)**'a bastığında, sonra da **İsteği dene**'ye bastığında perde arkasında olanlar:

1 Tarayıcı → sunucu

Tarayıcı, hedefini `/api/investigate` adresine gönderir. **Hedef API'ye henüz hiçbir şey gitmez.**

2 Sunucu ⇄ Gemini (araç döngüsü)

Sunucu hedefini ve araç listesini Gemini'ye yollar. Model araç ister, sunucu aracı çalıştırır (veritabanında anlam araması, işlem detayı), sonucu modele geri yollar. Bu 2–3 tur sürer; her adım ekrandaki “Araç izi” tablosuna ve `investigations` tablosuna yazılır.

3 Modelin önerisi (mor kutu)

“listProducts, status=on_sale, güven: yüksek” ve Türkçe gerekçe.

4 Kodun kontrolü (gri/mavi kutu)

İşlem dokümanda var mı? Model bu işlemi bir araç sonucunda gerçekten gördü mü? `on_sale` izin verilen değerlerden mi? Eksik zorunlu parametre var mı? Hepsisi geçerse URL program tarafından kurulur ve “hazır — gönderilmedi” yazar.

5 Sen onay verirsin

Bu işlemi incele ve dene → formu doldurur; **İsteği dene**'ye bastığında tarayıcı `/api/try` 'a yalnızca işlem adını ve parametreleri gönderir — **adres göndermez.**

6 Güvenli gönderim

Sunucu adresi dokümandan yeniden kurar, izin listesinde mi bakar, 5 sn zaman sınırı ve 64 KB boyut sınırıyla, yönlendirme takip etmeden gerçek GET isteğini atar.

7 Mağaza API'si cevap verir → yeşil kutu

`200 OK`, 8 ürün. Sonuç `try_runs` tablosuna kaydedilir; ekranda URL, durum kodu, süre, gerçek JSON ve kopyalanabilir bir TypeScript kod örneği görünür.

API Pathfinder

OpenAPI dokümanındaki GET işlemlerini bul, incele ve güvenli biçimde dene.

1. OpenAPI JSON içe aktar

```
{  "openapi": "3.0.3",  "info": {    "title": "Demo Store API",    "version": "1.0.0",    "description": "A small store API that ships with API Pathfinder for offline, repeatable demos."  },}
```

Demo dokümanı yükle

İçe aktar

#7 "Demo Store API": 4 GET işlemi kaydedildi, 4 embedding üretildi.
Atlandı: POST /products — Only GET operations are supported.

Dokümanı: #7 Demo Store API 1.0.0 — 4 işlem, 4 embedding

Hedefini yaz

Satışta olan ürünleri listele

Ham arama (yalnızca pgvector)

İncele (RAG)

İncele (Agent + araçlar)

Arama ve inceleme hedef API'ye istek göndermez.

ARAÇ İZİ — MODEL ÖNERDİ, SUNUCU ÇALIŞTIRDI (2 ARAÇ ADIMI, 3 MODEL TURU)

#	Modelin istediği çağrı	Kim çalıştırdı	Modele geri giden sonuç (özet)
1	search_endpoints({"query": "Satışta olan ürünleri listele"})	sunucu - 374 ms	listProducts (0.714), listCategories (0.643), getProductById (0.64), listOrders (0.632) ► tam sonuç
2	inspect_endpoint({"operation_id": "listProducts"})	sunucu - 2 ms	GET /products - params: status - executable: true ► tam sonuç

Durma nedeni: final_answer - En fazla 4 araç adımı. try_get_request modele verilmez.

MODEL YORUMU (GEMİNİ-3.5-FLASH-LİTE) — DOĞRULANMAMIŞ

ÇIKARIM - ARAÇ SONUÇLARINA GÖRE
Seçim: listProducts - güven: high
Kullanıcının "Satışta olan ürünleri listele" hedefi için "listProducts" operasyonu uygundur. Açıklamasında geçen "status" query parametresinin "on_sale" enum değeri, satışta/indirimde olan ürünleri filtrelemek için kullanılabilir.

KOD DOĞRULAMASI (DOKÜMANA GÖRE): HAZIR — GÖNDERİLMEDİ

İşlem: listProducts - Parametreler: {"status": "on_sale"}
Gönderilecek URL: http://localhost:3000/demo-api/products?status=on_sale

Bu işlemi incele ve dene →

Dokümandaki GET işlemleri

GET /products List products

GET /products/{id} Get a single product

GET /categories List product categories

GET /orders List orders çalıştırmaz

İncele ve dene

DOKÜMANDAN (OPENAPI)

GET /products - listProducts

List products

Returns all products. Use the optional status filter to return only products that are on sale, in stock, or out of stock.

Kaynak: #/paths/~1products/get

200 yanıt: A list of products. Shape: { count: integer, items: array of { id: integer, name: string, categoryId: integer, price: number, status: string } }

Parametre	Yer	Tip	Zorunlu	Açıklama
status	query	string (on_sale in_stock out_of_stock)	hayır	Filter by sale status. on_sale = currently discounted / on sale.

► Embedding için saklanan metin

status query

on_sale

İsteği dene

Gerçek GET isteği yalnızca bu düğmeyle gönderilir.

CANLI İSTEKTE GÖZLENEN

GET http://localhost:3000/demo-api/products?status=on_sale

Durum: 200 OK -41 ms - application/json

```
{  "count": 8,  "items": [    {      "id": 2,      "name": "Chef Knife",      "categoryId": 1,      "price": 17.25,      "status": "on_sale"    },    {      "id": 5,      "name": "Tea Kettle",      "categoryId": 1,      "price": 39.3,      "status": "on_sale"    },    ...  ]}
```

Uygulamanın yorumu: HTTP 200: the server accepted the request. This shows the call works, not that the data answers your goal — check the response.

TYPESCRIPT FETCH ÖRNEĞİ

```
const res = await fetch("http://localhost:3000/demo-api/products?status=on_sale", {  method: "GET",  headers: { Accept: "application/json" },});if (!res.ok) throw new Error("HTTP ${res.status}");const data: unknown = await res.json();console.log(data);
```

Senaryo 1'in gerçek ekranı: solda araç izi, model yorumu (mor) ve kod kontrolü; sağda dokümandan bilgiler (mavi) ve canlı yanıt (yeşil).

Güvenlik: sistem kendini nasıl koruyor?

Kullanıcının ya da yapay zekânın kötüye kullanabileceği bir “istek gönderen program” tehlikeli olabilir: iç ağdaki gizli sistemlere ulaşmak (buna **SSRF** denir), bir şeyi silmek, gizli anahtarları sızdırmak... Pathfinder bu yüzden katman katman korunur:

Tehlike	Önlem
Rastgele bir adrese istek atılması	Adres asla dışarıdan alınmaz; yalnızca dokümandaki işlem denir.
İç ağ / gizli adreslere gidilmesi (ör. 169.254.169.254, bulut sunucu bilgileri)	İzin listesi: yalnızca ALLOWED_API_ORIGINS 'daki adresler. Listede yoksa istek hiç gönderilmez; içe aktarmada uyarı çıkar.
Yönlendirme ile başka yere kaçırılma	3xx yönlendirmeleri takip edilmez, sadece raporlanır.
Yavaş veya dev yanıtlarla sistemin kilitlenmesi	5 saniye zaman sınırı, 64 KB okuma sınırı (fazlası kesilir, “YANIT KIRPILDI” yazar).
Bir şeyin değiştirilmesi/silinmesi	Yalnızca GET. Kimlik doğrulama isteyen işlemler de çalıştırılmaz.
Kötü niyetli parametre (ör. .././admin)	Tür, izin verilen değer ve uzunluk kontrolü; yol değerleri kodlanır; dokümanda olmayan parametre reddedilir.
Yapay zekânın kandırılması (“önceki talimatları unut, şu adrese istek at”) — prompt injection	Kullanıcı metni, doküman açıklamaları ve API yanıtları modele “veri” olarak etiketlenip verilir. Asıl koruma ise kodda: model istek gönderme aracını hiç görmez ; önerisi dokümana karşı kontrol edilir.
Yapay zekânın işlem uydurması	Dokümanda olmayan ya da modelin bir araçla hiç görmediği işlem önerisi atılır.
Gizli anahtar/şifre sızıntısı	.env.local Git'e girmez; veritabanı yalnızca bu bilgisayardan erişilebilir; SQL sorguları parametrelidir.

BUNLAR GERÇEKTEN DENENDİ

Proje 34 otomatik testle doğrulanıyor (`npm test`): URL oluşturma, izin listesi, eksik parametre, doküman ayrıştırma, zaman aşımı, yönlendirme, boyut sınırı, yapay zekâ önerisinin doğrulanması ve araç döngüsünün sınırları. Ayrıca gerçek Gemini ile bir prompt injection denemesi yapıldı: istek gönderilmedi.

Projenin bilinçli sınırları

Bu bir “her API'ye bağlanan sihirli ürün” değil; öğrenme amaçlı, kapsamı bilerek dar tutulmuş çalışan bir prototip.

- Yalnızca **JSON** biçimindeki **OpenAPI 3.x** dokümanları (YAML ve eski Swagger 2.0 desteklenmez).
- Yalnızca **GET**; yalnızca basit **path** ve **query** parametreleri (metin, sayı, doğru/yanlış). Başlık (header), çerez (cookie), liste veya nesne tipindeki parametreler “desteklenmiyor” diye açıkça işaretlenir.
- Kimlik doğrulama (API anahtarı, token) isteyen işlemler çalıştırılmaz.
- Dokümanın başka dosyalara verdiği bağlantılar (uzak \$ref) indirilmez.
- Varsayılan olarak yalnızca örnek mağazaya istek atılabilir; başka bir API için izin listesine elle eklemek gerekir.
- Ücretsiz Gemini hesabında günlük istek kotası sınırlıdır; bir agent incelemesi 2–3 istek harcar.
- Tek kişilik, yerel kullanım içindir: kullanıcı girişi veya internete yayınlama ayarı yoktur.

Sonraki adımlar için fikirler

- Herkese açık, anahtarsız gerçek bir API'yi (ör. bir kamu veri servisi) izin listesine ekleyip desteklemek.
- YAML desteği, liste tipinde parametreler, başlık parametreleri.
- Gelen yanıtın dokümandaki şekle uyup uymadığını otomatik kontrol etmek.
- Küçük bir ölçüm seti: “hedef → beklenen işlem” listesiyle RAG ve agent modlarının isabetini ölçmek.

Sözlük

Terim	Açıklama
Agent (ajan)	Bir aracın sonucuna göre sonraki adımını kendisi seçen, sınırlı sayıda adım atan yapay zekâ döngüsü.
API	Bir programın başka programlara açtığı, kuralları belli kapı.
API anahtarı	Bir servisi kullanma hakkını kanıtlayan gizli metin (şifre gibi).
Bağlam (context)	Yapay zekâyâ soruyla birlikte verilen bilgi.
Commit	Git'te kaydedilmiş bir değişiklik paketi ("kayıt noktası").
Durum kodu	HTTP yanıtının sonucunu anlatan 3 haneli sayı (200, 404...).
Embedding	Metnin anlamını temsil eden sayı listesi (burada 768 sayı).
Endpoint / uç nokta	API'deki tek bir işlemin adresi (ör. <code>GET /products</code>).
Enum	Bir parametre için izin verilen değerlerin sabit listesi (ör. <code>on_sale</code> , <code>in_stock</code> , <code>out_of_stock</code>).
Framework (çatı)	Uygulama yazmayı kolaylaştıran hazır iskelet (Next.js).
Function calling	Yapay zekânın bir aracın çalıştırılmasını <i>istemesi</i> ; çalıştıran uygulamadır.
GET / POST / PUT / PATCH / DELETE	HTTP yöntemleri: oku / oluştur / tamamen değiştir / kısmen değiştir / sil.
HTTP / HTTPS	Web'in iletişim dili / şifreli hali.
İstemci / sunucu	İsteyen taraf / cevap veren taraf.
JSON	Süslü ve köşeli parantezlerle yazılan, programların veri alışverişinde kullandığı metin biçimi.
Konteyner / imaj / volume	Docker'da çalışan kutu / kutunun kalıbı / kutu silinse de kalan kalıcı disk.
Kosinüs benzerliği	İki vektörün ne kadar aynı yöne baktığının ölçüsü; 1'e yakın = çok benzer.
localhost	"Bu bilgisayar".
LLM (dil modeli)	Metin okuyup metin üreten yapay zekâ (Gemini).
OpenAPI	API'lerin işlemlerini tarif eden standart doküman biçimi.
operationId	Bir API işleminin benzersiz adı.
Parametre (path / query)	İsteğe eklenen bilgi; yolun içine (<code>/products/17</code>) veya <code>?</code> 'den sonra (<code>?status=on_sale</code>).
pgvector	PostgreSQL için vektör saklama ve benzerlik arama eklentisi.
Port	Bir bilgisayardaki programların "kapı numarası".
PostgreSQL	Açık kaynaklı veritabanı sistemi.
Prompt injection	Yapay zekâyı, veri içine gizlenmiş talimatlarla kandırma girişi.
RAG	Önce ilgili bilgiyi getirip sonra yapay zekâyâ bu bilgiyle cevap üretme yöntemi.
\$ref	OpenAPI dokümanında "bu bilgi şurada" bağlantısı.
Route Handler	Next.js'te bir adrese gelen isteği karşılayan sunucu kodu (ör. <code>/api/try</code>).
SQL / parametrelili sorgu	Veritabanı sorgu dili / değerlerin sorguya yapılandırılmadan ayrı gönderildiği güvenli yazım.
SSRF	Bir sunucuyu, saldırganın erişemediği iç adreslere istek atmaya kandırma saldırısı.
Timeout	Bir işlemin en fazla ne kadar bekleneyeceği sınırı.
Vektör	Sıralı sayı listesi; burada bir metnin anlam koordinatları.

Projeyi ayağa kaldırma (adım adım)

A. İlk kurulum (bir kez yapılır)

Senin bilgisayarında bunların hepsi zaten yapıldı. Başka bir bilgisayarda sıfırdan kurmak için:

1 Gerekli programları kur

Node.js (LTS sürümü, nodejs.org — proje 24 ile test edildi), **Docker Desktop** (docker.com), **Git**. İstersen kod editörü olarak **VS Code**.

2 Projeyi bilgisayara al

GitHub'dan indir (`git clone ...`) ya da klasörü kopyala. Proje klasöründe bir terminal aç (VS Code: **Terminal** → **New Terminal**).

3 Hazır kod paketlerini indir

```
npm install
```

Bu komut `package.json` 'daki kütüphaneleri (Next.js, React, PostgreSQL sürücüsü, Gemini SDK...) `node_modules` klasörüne indirir.

4 Ayar dosyasını oluştur

```
copy .env.example .env.local
```

`.env.local` 'ı aç: `POSTGRES_PASSWORD` için bir şifre yaz ve **aynı şifreyi** `DATABASE_URL` içindeki `change-me` yerine koy.

5 Gemini API anahtarı al

aistudio.google.com/apikey adresinde "Create API key" → anahtarı kopyala → `.env.local` 'daki `GEMINI_API_KEY=` satırının sonuna boşluksuz yapıştır → kaydet. Anahtarı kimseyle paylaşma.

B. Her kullanımda

1 Docker Desktop'ı aç

Görev çubuğunda balina simgesi görünene ve "Engine running" olana kadar bekle.

2 Veritabanını başlat

```
npm run db:up
```

Beklenen: `Container pathfinder-db Started (veya Running)`. İstersen `docker ps` ile `pathfinder-db ... (healthy)` satırını gör.

3 (İsteğe bağlı) Testleri çalıştır

```
npm test
```

Beklenen son satır: `Tests 34 passed (34)`. Bu adım yapay zekâ kotası harcamaz.

4

Uygulamayı başlat

```
npm run dev
```

✓ Ready yazısını bekle. Bu terminal açık kaldığı sürece uygulama çalışır.

5

Tarayıcıda aç

```
http://localhost:3000
```

6

Kapatma

Uygulama: terminalde **Ctrl** + **C**. Veritabanı: `npm run db:down` (veriler silinmez).

Komut özeti

Komut	Ne yapar
<code>npm run db:up</code>	Veritabanı konteynerini başlatır
<code>npm run db:down</code>	Konteyneri durdurur (veri kalır)
<code>npm run db:logs</code>	Veritabanı kayıtlarını canlı gösterir (çıkış: Ctrl+C)
<code>npm run db:psql</code>	Veritabanına SQL yazabileceğin pencere açar (çıkış: \q)
<code>npm run db:reset</code>	Veritabanını tamamen siler ve boş tablolarla yeniden kurar
<code>npm run dev</code>	Uygulamayı geliştirme modunda başlatır (port 3000)
<code>npm test</code>	34 otomatik testi çalıştırır
<code>npm run typecheck</code>	Kodda tip hatası var mı kontrol eder

POWERSHELL UYARISI

Windows PowerShell'de `npm.ps1 cannot be loaded` hatası alırsan komutları `npm.cmd run dev` şeklinde yaz ya da Komut İstemi (cmd) kullan.

Ekran rehberi: her düğme ne işe yarar, bize ne sunar?

API Pathfinder
OpenAPI dokümanındaki GET işlemlerini bul, incele ve güvenli biçimde dene.

1. OpenAPI JSON içe aktar

OpenAPI 3.x JSON yapıştır

Demo dokümanı yükle JSON dosyası seç İçe aktar

Doküman: (seç)

Açılış ekranı

Bölüm “1. OpenAPI JSON içe aktar”

Seçenek	Ne yapar?	Bize ne sunar?
Metin kutusu	OpenAPI JSON'ını elle yapıştırabileceğin alan.	Herhangi bir API'nin dokümanını denemek.
Demo dokümanı yükle	Proje içindeki örnek mağaza dokümanını kutuya doldurur (henüz kaydetmez).	Hiçbir şey hazırlamadan hızlı başlangıç.
JSON dosyası seç	Bilgisayarından bir .json dosyası seçip içeriğini kutuya koyar (en fazla 1 MB).	Elindeki bir doküman dosyasını kolayca yüklemek.
İçe aktar	Dokümanı kontrol eder, GET işlemlerini çıkarır, veritabanına kaydeder ve her işlem için anlam vektörü üretir.	“4 GET işlemi kaydedildi, 4 embedding üretildi”, atlanan işlemler ve varsa uyarılar (ör. “sunucu izin listesinde değil”).
Doküman seçim kutusu	Daha önce içe aktarılmış dokümanlardan birini seçer.	Her seferinde yeniden içe aktarmadan çalışmak. Yanında işlem ve embedding sayısı yazar.
Eksik N embedding'i üret	Yalnızca bazı işlemlerin anlam vektörü yoksa görünür (ör. anahtar eklenmeden önce içe aktarıldıysa).	Arama yapılabilir hale getirir.

Sağ üst: “Dokümandaki GET işlemleri”

API Pathfinder

OpenAPI dokümanındaki GET işlemlerini bul, incele ve güvenli biçimde dene.

1. OpenAPI JSON içe aktar

OpenAPI 3.x JSON yapıştır

Demo dokümanı yükle JSON dosyası seç İçe aktar

Doküman: #1 Demo Store API 1.0.0 — 4 işlem, 4 embedding

Hedefini yaz

örn. Satışta olan ürünleri listele

Ham arama (yalnızca pgvector) İncele (RAG)

İncele (Agent + araçlar)

Arama ve inceleme hedef API'ye istek göndermez.

Dokümandaki GET işlemleri

- GET /products List products
- GET /products/{id} Get a single product
- GET /categories List product categories
- GET /orders List orders çalıştırmaz

Doküman seçildikten sonra: solda hedef alanı, sağda işlem listesi

Kayıtlı her GET işlemi listelenir. Bir satıra tıklamak o işlemi aşağıdaki **“İncele ve dene”** paneline açar — yapay zekâ kullanmadan, **elle** işlem seçip denemek için. Etiketler: **“çalıştırılmaz”** (kimlik doğrulama gibi bir engel var), **“embedding yok”** (henüz aranamaz).

Bölüm “Hedefini yaz”

Seçenek	Ne yapar?	Bize ne sunar?
Hedef kutusu	Ne yapmak istediğini doğal dille yaz (Türkçe olabilir). Enter agent incelemesini başlatır.	—
Ham arama (yalnızca pgvector)	Yapay zekâ yorumu olmadan, sadece anlam benzerliğine göre sıralama yapar.	Her işlemin benzerlik puanı ve mesafesi, embedding'e giden metin. “En yakın ≠ doğru” olduğunu görmek için.
İncele (RAG)	Sabit zincir: en yakın 4 işlemi bulur, modele verir, modelden gerekçeli öneri alır, kod kontrol eder.	“Getirilen bağlam” (modele tam olarak ne verildi), mor model yorumu, kod doğrulama kutusu.
İncele (Agent + araçlar)	Model araçları kendisi seçerek (en fazla 4 adım) araştırır.	“Araç izi” tablosu: modelin istediği her çağrı, sunucunun çalıştırması, süre, sonucu. Ardından model yorumu ve kod doğrulaması.

Bu üç düğmenin hiçbirisi hedef API'ye istek göndermez. Yapay zekâ kullanan ikisi Gemini kotası harcar (RAG ≈ 2, agent ≈ 3–4 çağrı).

Hedefini yaz

Siparişlerimi göster

Ham arama (yalnızca pgvector)

İncele (RAG)

İncele (Agent + araçlar)

Arama ve inceleme hedef API'ye istek göndermez.

► **Getirilen bağlam** — modele verilen 4 işlem (RAG'in "R'si")

MODEL YORUMU (GEMİNİ-3.5-FLASH-LİTE) — DOĞRULANMAMIŞ ÇIKARIM · SABİT ZİNCİR: ARA → CEVAPLA
Seçim: listOrders · güven: high
Kullanıcının amacı siparişlerini göstermektir. 'listOrders' operasyonu (GET /orders), belgelenmiş 'Returns the authenticated customer's orders' açıklamasıyla bu amaca doğrudan hizmet etmektedir.

KOD DOĞRULAMASI (DOKÜMANA GÖRE): ÇALIŞTIRILAMAZ
İşlem: listOrders · Parametreler: {}
Requires authentication (not supported in v1).

"Siparişlerimi göster" (RAG): model doğru işlemi buldu ama kod "çalıştırılmaz" dedi (kimlik doğrulama gerekiyor)

Sonuç kutuları: renkler ne anlatır?

Kutu	Kaynağı	Nasıl yorumlanmalı?
Model yorumu — doğrulanmamış çıkarım (mor)	Gemini	Bir öneri ve gerekçe. Yanılabilir; son söz değildir.
Kod doğrulaması: hazır — gönderilmedi	Pathfinder'ın kodu	Öneri dokümana uygun; gönderilecek URL hazır ama henüz gönderilmedi. Bu işlemi incele ve dene → formu doldurur.
eksik girdi / çalıştırılmaz / uygun işlem yok (sarı)	Pathfinder'ın kodu	Eksik zorunlu parametre var; ya işlem engelli (kimlik doğrulama, izin listesi); ya da dokümanda uygun işlem yok — uydurulmadı.
Dokümandan (OpenAPI) (mavi)	OpenAPI dokümanı	İşlemin kılavuzdaki gerçek tanımı: yol, açıklama, kaynak konumu, beklenen yanıt şekli, parametre tablosu.
Canlı istekte gözlenen (yeşil)	Gerçek HTTP yanıtı	2xx: istek başarılı. Veriyi kontrol et — 200 hedefin karşılandığını kanıtlamaz.
Canlı istekte gözlenen — HATA (kırmızı)	Gerçek HTTP yanıtı	4xx/5xx, zaman aşımı veya ağ hatası. Başarı gibi gösterilmez.

Panel “İncele ve dene”

Seçenek	Ne yapar?	Bize ne sunar?
Parametre alanları	İşlemin istediği bilgiler. Kırmızı * zorunlu demektir. İzin verilen değer listesi varsa açılır menü çıkar.	Yapay zekâ önerdiyse önceden dolu gelir; istediğin gibi değiştirebilirsin.
► Embedding için saklanan metin	Bu işlemin anlam aramasında kullanılan metni açar.	Aramanın neye göre yapıldığını görmek.
İsteği dene	Gerçek isteği gönderen tek düğme. Önce parametreleri kontrol eder; eksik/hatalıysa göndermez.	Gerçek URL, durum kodu, süre, içerik türü, gerçek yanıt (en fazla 64 KB), uygulamanın yorumu ve kopyalanabilir TypeScript fetch örneği . Her deneme <code>try_runs</code> tablosuna yazılır.

İncele ve dene

DOKÜMANDAN (OPENAPI)

GET /products/{id} · getProductById

Get a single product

Returns one product by its numeric ID, including name, price, category and sale status.

Kaynak: #/paths/~1products~1{id}/get

200 yanıt: The product. Shape: { id: integer, name: string, categoryId: integer, price: number, status: string }

Parametre	Yer	Tip	Zorunlu	Açıklama
id	path	integer	evet	Numeric product ID.

► Embedding için saklanan metin

id * path integer

İsteği dene

Gerçek GET isteği yalnızca bu düğmeyle gönderilir.

İSTEK GÖNDERİLMEDİ — GİRDİ GEREKLİ

Eksik zorunlu parametre: id (path, integer)

Zorunlu `id` boş: istek gönderilmedi

İncele ve dene

DOKÜMANDAN (OPENAPI)

GET /products/{id} · getProductById

Get a single product

Returns one product by its numeric ID, including name, price, category and sale status.

Kaynak: #/paths/~1products~1{id}/get

200 yanıt: The product. Shape: { id: integer, name: string, categoryId: integer, price: number, status: string }

Parametre	Yer	Tip	Zorunlu	Açıklama
id	path	integer	evet	Numeric product ID.

► Embedding için saklanan metin

id * path 999

İsteği dene

Gerçek GET isteği yalnızca bu düğmeyle gönderilir.

CANLI İSTEKTE GÖZLENEN — HATA

GET http://localhost:3000/demo-api/products/999

Durum: **404 Not Found** · 3895 ms · application/json

```
{
  "error": "not_found",
  "message": "No product with id 999"
}
```

Uygulamanın yorumu: HTTP 404: the API returned an error. This is NOT a successful result.

TYPESCRIPT FETCH ÖRNEĞİ

```
const res = await fetch("http://localhost:3000/demo-api/products/999", {
  method: "GET",
  headers: { Accept: "application/json" },
});
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data: unknown = await res.json();
console.log(data);
```

`id = 999`: gerçek 404, kırmızı hata kutusu

Deneme senaryoları ve sorun giderme

Önce **Demo dokümanı yükle** → **İçe aktar** (veya seçim kutusundan mevcut dokümanı seç). Sonra:

#	Hedef / işlem	Düğme	Beklenen	Neyi gösterir?
1	Satışta olan ürünleri listele	İncele (Agent) → Bu işlemi incele ve dene → İsteği dene	Araç izinde 2 adım; <code>listProducts</code> , <code>status=on_sale</code> ; 200 OK , 8 ürün	Uçtan uca akış, function calling
2	17 numaralı ürünü getir	İncele (Agent) → dene	<code>.../products/17</code> , 200, "Wireless Mouse"	Path parametresinin yerleşmesi
3	(Aynı form) <code>id</code> boş	İsteği dene	Sarı: "Eksik zorunlu parametre: id"	Eksik bilgide istek gönderilmez
4	(Aynı form) <code>id = 999</code>	İsteği dene	Kırmızı: 404 Not Found	Hata başarı gibi gösterilmez
5	Müşteri yorumlarını listele	Ham arama, sonra İncele (RAG)	Ham aramada yine 4 sonuç; RAG: "uygun işlem yok"	Uydurmama; en yakın ≠ doğru
6	Siparişlerimi göster	İncele (RAG)	<code>listOrders</code> bulunur ama "çalıştırılmaz"	Kimlik doğrulama sınırı
7	Merhaba, nasılsın?	İncele (Agent)	"Model hiç araç çağırmadı", "uygun işlem yok"	Dürüst sonuç

Yapay zekâ cevapları her seferinde birebir aynı kelimelerle gelmeyebilir; ancak seçilen işlem ve kodun kararı aynı olmalıdır. Her inceleme 5–15 saniye sürebilir.

Sorun giderme

Görülen	Sebebi ve çözüm
<code>ECONNREFUSED</code> / "Is the database running?"	Veritabanı kapalı. Docker Desktop'ı aç, <code>npm run db:up</code> .
<code>GEMINI_API_KEY is not set</code> (503)	Anahtar yok ya da sunucu anahtar eklendikten sonra yeniden başlatılmadı. <code>.env.local</code> 'u kontrol et, <code>npm run dev</code> 'i durdurup tekrar başlat.
<code>429 ... quota exceeded</code>	Gemini ücretsiz kotası doldu. Bekle ya da <code>GEMINI_MODEL</code> 'i değiştirip sunucuyu yeniden başlat.
<code>npm.ps1 cannot be loaded</code>	PowerShell betik kısıtlaması. <code>npm.cmd</code> ... kullan.
Port 3000 kullanımda	Başka bir <code>npm run dev</code> açık; o terminali kapat.
"Requests will not be sent: Origin ... is not in ALLOWED_API_ORIGINS"	Dokümanın sunucusu izin listesinde değil. Bilinçli bir güvenlik önlemi; gerçekten güveniyorsan adresi <code>.env.local</code> 'daki listeye ekleyip sunucuyu yeniden başlat.
Doküman listesinde aynı doküman birkaç kez	Her "İçe aktar" yeni kayıt açar. Temizlemek için <code>npm run db:reset</code> (tüm veriyi siler).

ÖZET

API Pathfinder; bir API kılavuzunu okuyup, anlam aramasıyla aday işlemleri bulan, yapay zekâyâ gerektelendiren, bu gerekteyi kodla doğrulayan ve yalnızca kullanıcının onayıyla, güvenlik sınırları içinde gerçek isteği gönderen küçük ama uçtan uca çalışan bir sistemdir. Her adımda "bilgi nereden geldi?" sorusunun cevabı ekranda görünür.